CS 202: IT Workshop I (End Sem Lab Exam)

Total Duration: 2 hours

Total Marks =65

Date: 25 Nov, 2025

Instructions:

1) Read the question carefully before start coding.
2) Classes are defined with instance members and methods. These are primary requirements during implementation of the classes. However, you are free to add additional member and methods as per your convenience.
3) Create your own test cases for evaluation. Creating nice test cases for better demonstration of your code is also a part of evaluation.
4) There are different components for marking. You will be offered marks based on the correctness of your code as well as your appropriate explanation.
5) **Don't copy from others or use any AI tool like chatgpt. We will check for plagiarism of your code. If your code is found to be similar with others, then immediately both will be offered zero in the lab examination.**
6) **Your internet should be off during the entire examination. Otherwise, your exam will be cancelled immediately.**
7) **During examination, you should open either of the two following screens in your system: the pdf containing question or your editor. In case, if you open any to other tab or screen, your exam may be cancelled.**

Question:

You need to implement a Multithreaded, Menu-Driven Online Cab Booking & Ride Allocation System in Java. The specifications for the problem are summarized below.

The following classes need to be implemented.

[Marks = 10+20+10+20+5=65]

A) **Abstract Class Cab**
Fields:
protected String cabId;
protected double baseFare; (Represents the fare per km of a cab)

Methods:

public abstract void *display():* Displays the cabId and the basefare of a cab
public double *getFare()* : Returns the total fare;

B) Classes for cab types

Create a class hierarchy: MiniCab → SedanCab → SUVCab. Each subclass applies a fare multiplier on baseFare, as follows

Cab Type	**Fare Multiplier**
Mini	baseFare × 1.0
Sedan	baseFare × 1.5
SUV	baseFare × 2.0

C) Class Booking

Fields:

private String *customerName*;
private Cab *cab*;
private double *distanceInKm*;
private double *totalFare*;

Methods:

public void *calculateFare();* // multiplier * distance. Check for discount while calculating fare.
public void *applyDiscount();* // 10% discount if distance > 20 km
public void *printDetails();* //Prints all the details related to the booking

D) Singleton Class BookingManager (Acts as the critical shared resource)

Only a single instance of the class will be created. It acts as the critical shared resource.

Fields:

private static BookingManager *instance* = null;
private ArrayList<Booking> *bookingList*;
private boolean *isDisplaying*;

Methods:

private *BookingManager()* {}
public static synchronized BookingManager *getInstance():* Returns the single instance of the BookingManager Class.

Note: There are three methods that are critical and needs synchronization, which are listed below. Use wait() and notifyAll() to coordinate access and avoid conflict. Details about the thread classes are given next.

addRequest() : This method is responsible for safely adding a new booking request to the shared list. If the system is currently displaying bookings (*isDisplaying == true*), the thread must **wait** until the display operation is finished to prevent inconsistent modification of the shared resource. Once safe, the booking is added and *notifyAll()* is called to wake up any waiting display threads**. Furthermore, discount on the booking will be automatically calculated during the booking.**

processRequests(): This method processes and displays all booking requests stored in the shared list. If the list is empty, the method calls wait() until at least one booking becomes available. During processing, the method sets *isDisplaying = true* to indicate exclusive access, ensures calculations like fare and discount are completed, prints all bookings, and then resets the state and calls *notifyAll()* so waiting threads can resume.

calculateSummary(): This method performs summary computations such as total bookings (option 5), highest fare (option 9), and average fare (option 10). It must only run after display processing is complete; therefore, it waits if *isDisplaying == true*. Once safe, it iterates over the list and prints summary results, ensuring consistency of shared data.

Add necessary methods for options 6, 7, 8. Give suitable names for the methods.

E) Classes for multi-threading: Two thread types share and access a common booking list. The specifications of the threads are as follows.

Thread Type	**Responsibility**
CustomerThread	The *CustomerThread* represents a customer placing a cab booking request. It calls *addRequest*() of BookingManager to safely

	insert the booking into the shared list. If the system is currently displaying bookings, the thread waits until display operations complete. This ensures that booking addition and display do not occur simultaneously, maintaining consistency and preventing concurrent modification errors.
DisplayThread	The *DisplayThread* is responsible for processing and displaying all current booking requests. When executed, it invokes *processRequests*() in the BookingManager, which waits if no bookings exist. While processing, it sets a flag (*isDisplaying*) to block new booking additions, displays all confirmed bookings, and then initiates the summary calculation (*calculateSummary*()), such as highest fare and average fare. Once done, it releases control using notifyAll(), allowing waiting customer threads to resume.

F) **Driver Class (Used for creating menu):** The Driver class must implement a menu with the following options for the user.

1**. Book Mini Cab:** Book a mini cab using customer thread. Print information related to booking

2**. Book Sedan**: Book a Sedan cab using customer thread. Print information related to booking

3**. Book SUV:** Book a SUV cab using customer thread. Print information related to booking

4**. Show all bookings:** Displays all the booking with the help of the display thread

5. **Show total booking count:** Shows the total number of current bookings.

6. **Search booking by customer name:** When a customer name is passed, the booking information related to him should be retrieved and printed.

7. **Sort bookings by fare (Ascending)**: Print the all the current booking information in ascending order of fare.

8**. Discount Receivers** (distance > 20 km): Show the customers with their original fair and discounted fair.

9. **Show highest fare ride**: Show the booking with highest fair ride

10. **View average fare:** Prints the average fare of all the current bookings.

11. **Exit:** Exit out of the system